

Experiment 'p_6_2_e_remove_middle_array' Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_6_2_e_remove_middle_array

Natural language statement: Write a method to remove the middle element of an array if the array length is odd, or the middle two elements if the length is even.

Method signature: p_6_2_e_remove_middle_array(inputs: array<int>) returns (result: array<int>)

Ensures

- `result.Length == if inputs.Length % 2 == 1 then inputs.Length - 1 else inputs.Length - 2`
- `inputs.Length % 2 == 1 ==> forall i :: 0 <= i < inputs.Length / 2 ==> result[i] == inputs[i] && forall j :: inputs.Length / 2 <= j < result.Length ==> result[j] == inputs[j + 1]`
- `inputs.Length % 2 == 0 ==> forall i :: 0 <= i < inputs.Length / 2 - 1 ==> result[i] == inputs[i] && forall j :: inputs.Length / 2 - 1 <= j < result.Length ==> result[j] == inputs[j + 2]`

Requires

- `inputs.Length >= 1`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to remove the middle element of an array if the array length is odd, or
↪ the middle two elements if the length is even.

The signature should be:

p_6_2_e_remove_middle_array(inputs: array<int>) returns (result: array<int>)

The method should respect the following contract:

requires inputs.Length >= 1, ensures result.Length == if inputs.Length % 2 == 1 then
↪ inputs.Length - 1 else inputs.Length - 2, ensures inputs.Length % 2 == 1 ==>
↪ forall i :: 0 <= i < inputs.Length / 2 ==> result[i] == inputs[i] && forall j ::
↪ inputs.Length / 2 <= j < result.Length ==> result[j] == inputs[j + 1], ensures
↪ inputs.Length % 2 == 0 ==> forall i :: 0 <= i < inputs.Length / 2 - 1 ==> result[i
↪] == inputs[i] && forall j :: inputs.Length / 2 - 1 <= j < result.Length ==>
↪ result[j] == inputs[j + 2]

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn0KpC5ByPqUurB7vQf6RrEhGO7Lu

Model: gpt-5.5-2026-04-23

Created at: 1780571731

Finish reason: stop

Usage: 580 tokens in, and 5475 tokens out

```
{
  var n := inputs.Length;
  if n % 2 == 1 {
    var mid := n / 2;
    assert n == (n / 2) * 2 + n % 2;
    assert n == mid * 2 + 1;
    assert 0 <= mid;
    assert mid <= n - 1;

    result := new int[n - 1];

    var i := 0;
    while i < mid
      invariant 0 <= i <= mid
```

```

    invariant result.Length == n - 1
    invariant n == inputs.Length
    invariant forall k :: 0 <= k < i ==> result[k] == inputs[k]
  {
    result[i] := inputs[i];
    i := i + 1;
  }

var j := mid;
while j < result.Length
  invariant mid <= result.Length
  invariant mid <= j <= result.Length
  invariant result.Length == n - 1
  invariant n == inputs.Length
  invariant forall k :: 0 <= k < mid ==> result[k] == inputs[k]
  invariant forall k :: mid <= k < j ==> result[k] == inputs[k + 1]
  {
    result[j] := inputs[j + 1];
    j := j + 1;
  }

assert mid == inputs.Length / 2;
assert forall k :: 0 <= k < inputs.Length / 2 ==> result[k] == inputs[k];
assert forall k :: inputs.Length / 2 <= k < result.Length ==> result[k] == inputs[k +
  ↪ 1];
} else {
  assert 0 <= n % 2 < 2;
  assert n % 2 == 0;
  if n == 1 {
    assert n % 2 == 1;
    assert false;
  }
  assert n >= 2;

  var mid := n / 2;
  assert n == (n / 2) * 2 + n % 2;
  assert n == mid * 2;
  assert 1 <= mid;

  var split := mid - 1;
  assert 0 <= split;
  assert split <= n - 2;

  result := new int[n - 2];

  var i := 0;
  while i < split
    invariant 0 <= i <= split
    invariant result.Length == n - 2
    invariant n == inputs.Length
    invariant forall k :: 0 <= k < i ==> result[k] == inputs[k]

```

```

{
  result[i] := inputs[i];
  i := i + 1;
}

var j := split;
while j < result.Length
  invariant split <= result.Length
  invariant split <= j <= result.Length
  invariant result.Length == n - 2
  invariant n == inputs.Length
  invariant forall k :: 0 <= k < split ==> result[k] == inputs[k]
  invariant forall k :: split <= k < j ==> result[k] == inputs[k + 2]
  {
    result[j] := inputs[j + 2];
    j := j + 1;
  }

  assert split == inputs.Length / 2 - 1;
  assert forall k :: 0 <= k < inputs.Length / 2 - 1 ==> result[k] == inputs[k];
  assert forall k :: inputs.Length / 2 - 1 <= k < result.Length ==> result[k] == inputs[
    ↪ k + 2];
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_6_2_e_remove_middle_array/program.dfy(13,56): Error: index out of range
```

```

|
13 | result := new int[n - 1](k => if k < mid then inputs[k] else inputs[k + 1]);
| ^

```

```
problem_data/p_6_2_e_remove_middle_array/program.dfy(13,56): Error: insufficient reads
```

```

↪ clause to read array element; Consider extracting inputs[k] to a local variable
↪ before the lambda expression, or adding 'reads inputs' in the enclosing lambda
↪ specification for resolution

```

```

|
13 | result := new int[n - 1](k => if k < mid then inputs[k] else inputs[k + 1]);
| ^

```

```
problem_data/p_6_2_e_remove_middle_array/program.dfy(13,71): Error: index out of range
```

```

|
13 | result := new int[n - 1](k => if k < mid then inputs[k] else inputs[k + 1]);
| ^

```

```
problem_data/p_6_2_e_remove_middle_array/program.dfy(13,71): Error: insufficient reads
```

```

↪ clause to read array element; Consider extracting inputs[k + 1] to a local
↪ variable before the lambda expression, or adding 'reads inputs' in the enclosing
↪ lambda specification for resolution

```

```

|
13 | result := new int[n - 1](k => if k < mid then inputs[k] else inputs[k + 1]);
| ^

problem_data/p_6_2_e_remove_middle_array/program.dfy(53,57): Error: index out of range
|
53 | result := new int[n - 2](k => if k < left then inputs[k] else inputs[k + 2]);
| ^

```

Dafny program verifier finished with 1 verified, 5 errors

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn0NkK1lLwU54zpC4IRrwWdzyTir3

Model: gpt-5.5-2026-04-23

Created at: 1780571912

Finish reason: stop

Usage: 2046 tokens in, and 6903 tokens out

```

{
  var n := inputs.Length;

  if n % 2 == 1 {
    var mid := n / 2;
    assert 0 <= n % 2 < 2;
    assert n == (n / 2) * 2 + n % 2;
    assert n == mid * 2 + 1;
    assert 0 <= mid;
    assert mid < n;

    result := new int[n - 1];
    assert result != inputs;
    assert result.Length == n - 1;
    assert result.Length == mid * 2;
    assert mid <= result.Length;

    var i := 0;
    while i < mid
      invariant 0 <= i <= mid
      invariant n == inputs.Length
      invariant result != inputs
      invariant result.Length == n - 1
      invariant mid <= result.Length
      invariant mid < n
      invariant forall k :: 0 <= k < i ==> result[k] == inputs[k]

```

```

{
  assert 0 <= i < result.Length;
  assert 0 <= i < inputs.Length;
  result[i] := inputs[i];
  i := i + 1;
}

var j := mid;
while j < result.Length
  invariant mid <= j <= result.Length
  invariant n == inputs.Length
  invariant result != inputs
  invariant result.Length == n - 1
  invariant mid <= result.Length
  invariant forall k :: 0 <= k < mid ==> result[k] == inputs[k]
  invariant forall k :: mid <= k < j ==> result[k] == inputs[k + 1]
{
  assert 0 <= j < result.Length;
  assert 0 <= j + 1 < inputs.Length;
  result[j] := inputs[j + 1];
  j := j + 1;
}

assert mid == inputs.Length / 2;
assert result.Length == inputs.Length - 1;
assert result.Length == (if inputs.Length % 2 == 1 then inputs.Length - 1 else inputs.
  ↪ Length - 2);
assert forall k :: 0 <= k < inputs.Length / 2 ==> result[k] == inputs[k];
assert forall k :: inputs.Length / 2 <= k < result.Length ==> result[k] == inputs[k +
  ↪ 1];
assert forall k :: 0 <= k < inputs.Length / 2 ==> result[k] == inputs[k] && (forall t
  ↪ :: inputs.Length / 2 <= t < result.Length ==> result[t] == inputs[t + 1]);
} else {
  assert n % 2 != 1;
  assert 0 <= n % 2 < 2;
  assert n % 2 == 0;
  if n == 1 {
    assert n % 2 == 1;
    assert false;
  }
  assert n >= 2;

  var mid := n / 2;
  assert n == (n / 2) * 2 + n % 2;
  assert n == mid * 2;
  assert 1 <= mid;

  var left := mid - 1;
  assert 0 <= left;
  assert mid == left + 1;

```

```

result := new int[n - 2];
assert result != inputs;
assert result.Length == n - 2;
assert result.Length == left * 2;
assert left <= result.Length;

var i := 0;
while i < left
    invariant 0 <= i <= left
    invariant n == inputs.Length
    invariant result != inputs
    invariant result.Length == n - 2
    invariant left <= result.Length
    invariant forall k :: 0 <= k < i ==> result[k] == inputs[k]
{
    assert 0 <= i < result.Length;
    assert 0 <= i < inputs.Length;
    result[i] := inputs[i];
    i := i + 1;
}

var j := left;
while j < result.Length
    invariant left <= j <= result.Length
    invariant n == inputs.Length
    invariant result != inputs
    invariant result.Length == n - 2
    invariant left <= result.Length
    invariant forall k :: 0 <= k < left ==> result[k] == inputs[k]
    invariant forall k :: left <= k < j ==> result[k] == inputs[k + 2]
{
    assert 0 <= j < result.Length;
    assert 0 <= j + 2 < inputs.Length;
    result[j] := inputs[j + 2];
    j := j + 1;
}

assert left == inputs.Length / 2 - 1;
assert result.Length == inputs.Length - 2;
assert result.Length == (if inputs.Length % 2 == 1 then inputs.Length - 1 else inputs.
    ↪ Length - 2);
assert forall k :: 0 <= k < inputs.Length / 2 - 1 ==> result[k] == inputs[k];
assert forall k :: inputs.Length / 2 - 1 <= k < result.Length ==> result[k] == inputs[
    ↪ k + 2];
assert forall k :: 0 <= k < inputs.Length / 2 - 1 ==> result[k] == inputs[k] && (
    ↪ forall t :: inputs.Length / 2 - 1 <= t < result.Length ==> result[t] == inputs[
    ↪ t + 2]);
}
}

```

Final Program

```
method p_6_2_e_remove_middle_array(inputs: array<int>) returns (result: array<int>)
  requires inputs.Length >= 1
  ensures result.Length == if inputs.Length % 2 == 1 then inputs.Length - 1 else
    ↪ inputs.Length - 2
  ensures inputs.Length % 2 == 1 ==> forall i :: 0 <= i < inputs.Length / 2 ==>
    ↪ result[i] == inputs[i] && forall j :: inputs.Length / 2 <= j < result.
    ↪ Length ==> result[j] == inputs[j + 1]
  ensures inputs.Length % 2 == 0 ==> forall i :: 0 <= i < inputs.Length / 2 - 1 ==>
    ↪ result[i] == inputs[i] && forall j :: inputs.Length / 2 - 1 <= j < result.
    ↪ Length ==> result[j] == inputs[j + 2]
{
  var n := inputs.Length;

  if n % 2 == 1 {
    var mid := n / 2;
    assert 0 <= n % 2 < 2;
    assert n == (n / 2) * 2 + n % 2;
    assert n == mid * 2 + 1;
    assert 0 <= mid;
    assert mid < n;

    result := new int[n - 1];
    assert result != inputs;
    assert result.Length == n - 1;
    assert result.Length == mid * 2;
    assert mid <= result.Length;

    var i := 0;
    while i < mid
      invariant 0 <= i <= mid
      invariant n == inputs.Length
      invariant result != inputs
      invariant result.Length == n - 1
      invariant mid <= result.Length
      invariant mid < n
      invariant forall k :: 0 <= k < i ==> result[k] == inputs[k]
    {
      assert 0 <= i < result.Length;
      assert 0 <= i < inputs.Length;
      result[i] := inputs[i];
      i := i + 1;
    }

    var j := mid;
    while j < result.Length
      invariant mid <= j <= result.Length
      invariant n == inputs.Length
      invariant result != inputs
      invariant result.Length == n - 1
      invariant mid <= result.Length
```

```

    invariant forall k :: 0 <= k < mid ==> result[k] == inputs[k]
    invariant forall k :: mid <= k < j ==> result[k] == inputs[k + 1]
  {
    assert 0 <= j < result.Length;
    assert 0 <= j + 1 < inputs.Length;
    result[j] := inputs[j + 1];
    j := j + 1;
  }

  assert mid == inputs.Length / 2;
  assert result.Length == inputs.Length - 1;
  assert result.Length == (if inputs.Length % 2 == 1 then inputs.Length - 1 else inputs.
    ↪ Length - 2);
  assert forall k :: 0 <= k < inputs.Length / 2 ==> result[k] == inputs[k];
  assert forall k :: inputs.Length / 2 <= k < result.Length ==> result[k] == inputs[k +
    ↪ 1];
  assert forall k :: 0 <= k < inputs.Length / 2 ==> result[k] == inputs[k] && (forall t
    ↪ :: inputs.Length / 2 <= t < result.Length ==> result[t] == inputs[t + 1]);
} else {
  assert n % 2 != 1;
  assert 0 <= n % 2 < 2;
  assert n % 2 == 0;
  if n == 1 {
    assert n % 2 == 1;
    assert false;
  }
  assert n >= 2;

  var mid := n / 2;
  assert n == (n / 2) * 2 + n % 2;
  assert n == mid * 2;
  assert 1 <= mid;

  var left := mid - 1;
  assert 0 <= left;
  assert mid == left + 1;

  result := new int[n - 2];
  assert result != inputs;
  assert result.Length == n - 2;
  assert result.Length == left * 2;
  assert left <= result.Length;

  var i := 0;
  while i < left
    invariant 0 <= i <= left
    invariant n == inputs.Length
    invariant result != inputs
    invariant result.Length == n - 2
    invariant left <= result.Length
    invariant forall k :: 0 <= k < i ==> result[k] == inputs[k]

```

```

{
  assert 0 <= i < result.Length;
  assert 0 <= i < inputs.Length;
  result[i] := inputs[i];
  i := i + 1;
}

var j := left;
while j < result.Length
  invariant left <= j <= result.Length
  invariant n == inputs.Length
  invariant result != inputs
  invariant result.Length == n - 2
  invariant left <= result.Length
  invariant forall k :: 0 <= k < left ==> result[k] == inputs[k]
  invariant forall k :: left <= k < j ==> result[k] == inputs[k + 2]
{
  assert 0 <= j < result.Length;
  assert 0 <= j + 2 < inputs.Length;
  result[j] := inputs[j + 2];
  j := j + 1;
}

assert left == inputs.Length / 2 - 1;
assert result.Length == inputs.Length - 2;
assert result.Length == (if inputs.Length % 2 == 1 then inputs.Length - 1 else inputs.
  ↪ Length - 2);
assert forall k :: 0 <= k < inputs.Length / 2 - 1 ==> result[k] == inputs[k];
assert forall k :: inputs.Length / 2 - 1 <= k < result.Length ==> result[k] == inputs[
  ↪ k + 2];
assert forall k :: 0 <= k < inputs.Length / 2 - 1 ==> result[k] == inputs[k] && (
  ↪ forall t :: inputs.Length / 2 - 1 <= t < result.Length ==> result[t] == inputs[
  ↪ t + 2]);
}
}

```

Total Token Usage

Input tokens: 2626

Output tokens: 12378

Reasoning tokens: 10209

Sum of 'total tokens': 15004

Experiment Timings

Overall Experiment started at 1780571730484, ended at 1780572086445, lasting 355961ms (355.96 seconds)

Iteration #1 started at 1780571730484, ended at 1780571910111, lasting 179627ms (179.63 seconds)

Iteration #2 started at 1780571910111, ended at 1780572086445, lasting 176334ms (176.33 seconds)